An Improved Duplication Strategy for Scheduling Precedence Constrained Graphs in Multiprocessor Systems

Savina Bansal, *Student Member*, *IEEE*, Padam Kumar, *Member*, *IEEE*, and Kuldip Singh

Abstract—Scheduling precedence constrained task graphs, with or without duplication, is one of the most challenging NP-complete problems in parallel and distributed computing systems. Duplication heuristics are more effective, in general, for fine grain tasks graphs and for networks with high communication latencies. However, most of the available duplication algorithms are designed under the assumption of unbounded availability of fully connected processors, and lie in high complexity range. Low complexity optimal duplication algorithms work under restricted cost and/or shape parameters for the task graphs. Further, the required number of processors grows in proportion to the task-graph size significantly. An improved duplication strategy is proposed that works for arbitrary task graphs, with a limited number of interconnection-constrained processors. Unlike most other algorithms that replicate all possible parents/ancestors of a given task, the proposed algorithm tends to avoid redundant duplications and duplicates the nodes selectively, only if it helps in improving the performance. This results in lower duplications and also lower time and space complexity. Simulation results are presented for clique and an interconnection-constrained network topology with random and regular benchmark task graph suites, representing a variety of parallel numerical applications. Performance, in terms of normalized schedule length and efficiency, is compared with some of the well-known and recently proposed algorithms. The suggested algorithm turns out to be most efficient, as it generates better or comparable schedules with remarkably less processor consumption.

Index Terms—Algorithm, distributed computing, interconnection network, multiprocessor scheduling.

1 Introduction

Distributed memory machines have become quite popular in the recent past, due to advances in VLSI technology, low latency communication interfaces, networking protocols, and efficient routing algorithms. Task scheduling is one of the most challenging problems in parallel and distributed computing systems. In deterministic scheduling, the application task graph also known as the macro dataflow graph, is generally represented by a directed acyclic graph (DAG) that is assumed to be known a priori. Efficient scheduling of these task graphs plays a crucial role in extracting maximum possible throughput from parallel architectures.

Scheduling is a well-known NP-complete problem and polynomial time solutions are known to exist in very limited cases under highly restricted constraints that, at times, are too ideal to suit the real world problems [22], [7], [29], [12], [30]. As a result, heuristic-based algorithms developed over the last four decades tend to sacrifice optimality for the sake of efficiency [13], [33], [29], [19], [2], [14], [34], [35], [18], [27]. Interprocess communication continues to impose performance limitations on scheduling algorithms.

- *P. Kumar and K. Singh are with the Department of Electronics and Computer Engineering, Indian Institute of Technology, Roorkee (Uttranchal) -247 667, India. E-mail: {padamfec, kds56fec}@iitr.ernet.in.*
- *S. Bansal is on study leave from GZS College of Engineering and Technology, Bathinda (Punjab) - 151 001, India. E-mail: saavidec@iitr.ernet.in.*

Manuscript received 8 Apr. 2002; revised 24 Nov. 2002; accepted 24 Feb. 2003.
For information on obtaining reprints of this article, please send e-mail to: tpds@computer.org, and reference IEEECS Log Number 116294.

Scheduling heuristics may be broadly classified as *list-based*, *clustering-based*, and *duplication-based* heuristics. In the classical list scheduling technique, a task sequence satisfying precedence constraints and, generally, based on some priority, is generated statically or dynamically. The priorities are based on computation and communication costs in the task graph. Some of the frequently used priority terms for a given task are: *s-level* (computation cost along the longest directed path from the concerned task to the exit task in the given DAG), *b-level* (same as *s-level*, but considering communication costs as well), *t-level* (computation and communication costs along the longest directed path from the entry node to the concerned node excluding its computation cost), *as-late-as-possible* bindings, and *dynamic level* [33], [29], [19], [14], [34], [16], [17]. The highest priority task among the ready tasks (whose precedence have been met and are ready for scheduling) is next chosen, followed by the selection of the most suitable processor to accommodate it. Performance of list scheduling algorithms tends to suffer due to min-max problem and deteriorates substantially for fine grain task graphs having high communication to computation cost ratio (CCR) [29], [31].

Clustering-based algorithms try to schedule heavily communicating tasks onto the same processor, even if other processors are available, thereby trading off parallelism with interprocess communication. It is also known as two phase scheduling. In the first phase, heavily communicating tasks are grouped into a set of clusters (unbounded) using linear or nonlinear clustering heuristics [13], [15], [37] [36] and, in the second phase, clusters are mapped onto the set of available processors using communication sensitive or

1045-9219/03/$17.00 © 2003 IEEE Published by the IEEE Computer Society

insensitive heuristics [11]. Complexity of clustering algorithms tends to be lower in comparison to list-based techniques because during the clustering phase, the former is not required to work on the limited processors constraint as is generally the case with the latter, and during the mapping phase, low complexity load balancing heuristics may be employed in clustering-based algorithms. However, comparison in terms of quality of schedule generated is left open [23].

Duplication of nodes on more than one processor, to reduce the waiting time of the dependent tasks, is an interesting approach that has been blended with both list and clustering-based techniques by various researchers [2], [24], [23], [25], [3], [4]. List or clustering algorithms with duplication tend to perform better than nonduplication algorithms [22], [24]. However, the improvement in performance usually comes at the cost of increase in complexity, as these algorithms have to carry out backward search for duplicating the parents/ancestors of a node.

Most of the duplication algorithms like the Critical Path-based Full Duplication algorithm (CPFD) [2], Duplication First Reduction Next algorithm DFRN [26], Bottom up Top down Duplication Heuristic BTDH [6], Duplication Scheduling Heuristic DSH [21], and algorithms proposed by Papadimitriou et al. [24] and Palis et al. [23], try to duplicate all possible ancestors of a given node to improve performance and, in the process, become quite complex and resource consuming [2]. Moreover, the required number of processors grows significantly with the task graph size, thereby limiting their practical utility for the problems of large sizes.

Further, we observed that in many duplication algorithms, all the duplication done is not useful [3]. Redundant duplications, which do not contribute toward performance improvement, are also present leading to superfluous consumption of resources. Our objective is to develop an efficient duplication algorithm that extends maximum benefit of limited duplication to any list-based scheme, so as to improve its performance for fine grain task graphs, and still works with limited number of interconnection-constrained processors. No results have so far been reported, as far as we know, for duplication algorithms with interconnection constrained networks.

The work is organized as follows: Section 2 formulates the scheduling problem. Section 3 presents the related work and motivation for the suggested Selective Duplication (SD) algorithm that has been expounded in the sections. Performance results in comparison with some of the best available algorithms are presented in the following section. Section 5 discusses and concludes the work.

2 Scheduling Problem Formulation

Scheduling for a parallel processing system relates to assigning the tasks of a parallel program to the set of processors, and suggesting time order of execution on them while maintaining precedence amongst the tasks and reducing overall execution time (makespan) of the program. A quadruple Q represent the task model as $\mathrm{Q}(\mathrm{T}, \mathbf{R}, [c_{ij}], [w(t)])$, where $\mathrm{T} = \{t_i : i = 1, 2 \ldots, v\}$ is a set of "v" tasks, "$\mathbf{R}$" represents a relation that defines a partial order on the task set T such that

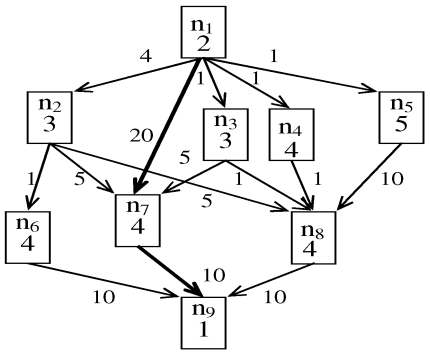

Fig. 1. A simple DAG representing a precedence constrained task graph.

if $\mathrm{t_i}\ \mathbf{R}\ \mathrm{t_j}$, then task $\mathrm{t_i}$ must finish before $\mathrm{t_j}$ can start execution; $[c_{ij}]$ is a $v \times v$ matrix giving the amount of dataflow (also referred as communication cost) between task $\mathrm{t_i}$ and $\mathrm{t_j}$ for $1 \leq i, j \leq v$; and $[w(t)]$ is a "v" vector of computation requirements, i.e., $\mathrm{w(t_i)} > 0$ implies a number of instructions required to compute task $\mathrm{t_i}$, $1 \leq i \leq v$. The task model can be visualized by a DAG as shown in Fig. 1 (example DAG taken from [2]), with vertices representing the tasks, edges the dataflow paths, and the corresponding weights on them the computation and communication costs respectively. The term node and task has been used interchangeably in the paper. Candidate node and candidate processor refer to the node/processor that is under consideration for scheduling at a given time.

The target machine architecture is represented by a three-tuple $\mathrm{M} = (\mathrm{P}, [\mathrm{L_{ij}}], [\mathrm{h_{ij}}]); \mathrm{P} = \{\mathrm{p_1}, \mathrm{p_2} \ldots .., \mathrm{p_p}\}$ is a set of "p" homogeneous processors; $[\mathrm{L_{ij}}]$ is a $\mathrm{p} \times \mathrm{p}$ matrix describing interconnection network topology with a 1/0 indicating the presence/absence of a direct link between $\mathrm{p_i}$ and $\mathrm{p_j}$; and $[\mathrm{h_{ij}}]$ is a $\mathrm{p} \times \mathrm{p}$ matrix giving minimum distance in number of hops between processor $\mathrm{p_i}$ and $\mathrm{p_j}$. Communication links between processors are assumed contention free to accommodate the deterministic nature of scheduling, and communication between them is through I/O channels, thereby allowing concurrent computation and communication. Further, the communication overhead between two tasks scheduled on the same processor is taken as zero.

The objective of scheduling is to allocate each task in the task graph to a processor in the processor network, and assignment of a start time so that the makespan or schedule length (SL) (as defined (1)) is minimized. The term $\mathrm{F_{i,j}}$ refers to finish time of task $\mathrm{t_i}$ on processor $\mathrm{p_j}$.

$$SL = \max\{\mathrm{F}_{i,j}\} \quad \forall 1 \leq i \leq v \quad \text{and} \quad \forall 1 \leq j \leq p. \qquad (1)$$

3 Selective Duplication Algorithm

3.1 Related Work and Motivation

The scheduling problem, with or without duplication, has been shown to be NP complete [24] for arbitrary DAGs. However, polynomial time approximation algorithms with known performance ratio have been suggested from time to time [22], [24], [23], [30]. Algorithms that attempt to work out the problem optimally do so by restricting cost or shape parameters in the task graph [9], [8], [25], [36]. Many duplication algorithms perform without any guaranteed

performance ratio for arbitrary DAGs [2], [26], however, their optimality for some restricted task graph types has been proved in literature.

For unit execution and unit communication time (UET/UCT) arbitrary DAGs, duplication-based heuristics have been shown to give performance bounds better than nonduplication-based heuristics [22]. The best known polynomial time approximation algorithm (PY) for arbitrary DAGs was suggested by Papadimitriou et al. [24] with a time complexity of $O(v^2(v\log v+e))$ ("v" and "e" being the number of nodes and edges in the given DAG, respectively). The PY algorithm generates schedules whose makespan is, at the most, twice optimal, provided task duplication is allowed and an unbounded number of processors is available to host the generated clusters. Palis et al. [23] suggested the Greedy version of this algorithm (referred as GPY algorithm in this paper) with the same performance ratio and with reduced time complexity as $O(v(v\log v+e))$.

For task graphs with small communication delays, Colin et al. [8] proposed an optimal duplication algorithm (referred as LB here, because of the lower bound concept used in it), provided the following cost constraint gets satisfied in the DAG:

$$\min_{t_i\in pred(t_j)} w(t_i) \geq \max_{t_i\in pred(t_j)} c_{ij},$$

for any join task $t_j \in T$.

Here, pred(t_j) corresponds to the set of immediate predecessors of task t_j in the DAG.

Darbha and Agrawal [9] improved upon the LB algorithm's optimality criterion to include task graphs with a wider computation and communication cost parameters in their Task Duplication based Scheduling algorithm (TDS) with $O(v^2)$ time complexity. The optimality criterion of TDS algorithm is as follows: If t_j is a join node with immediate predecessors t_i and t_k such that the data arrival times ($ect(t_i)+c_{ij}$) from them are the highest and the second highest among all predecessors, then the following inequalities should be satisfied for optimal results:

$$\begin{aligned} &\text{if } est(t_i) \geq est(t_k), \quad w(t_k) \geq c_{kj} \\ &\text{if } est(t_i) < est(t_k), \quad w(t_k) \geq (c_{kj} + est(t_k) - est(t_i)), \end{aligned}$$

where

$$est(t_j) = \begin{cases} 0 & \text{if pred(j)} = 0 \\ \max\{ect(t_i), (ect(t_k)+c_{kj})\} & \text{otherwise, and} \end{cases}$$

$ect(t_j) = est(t_j) + w(t_j)$.

LB and TDS algorithms generate linear clusters, thereby limiting parallelism available in the task graph. In a recent work, Park et al. [25] have enhanced the TDS optimality criterion by merging linear clusters of immediate parents under an optimal cluster merging cost constraint (see [25] for details), thereby generating optimal nonlinear clusters. We refer to their algorithm as the Modified TDS (MTDS) algorithm in this paper. The complexity of the MTDS algorithm is $O(v^2 d_{\max})$, and the complexity of testing optimality condition is $O(v^3 d_{\max})$ ($d_{\max}$ being the maximum number of immediate predecessors of nodes).

To study the practicability of these algorithms on arbitrary DAGs (as LB, PY, GPY, and MTDS algorithms have been studied theoretically only), we first coded these algorithms and, then, simulated their performance on random and regular benchmark arbitrary task graph suites. The number of clusters generated by all these algorithms turns out to be quite high. Consequently, the need of an algorithm that exploits benefits of duplication within limited resources is greatly felt. Here, we are proposing a duplication-based heuristic algorithm that tends to retain the best features of duplication and still works with a limited number of interconnection constrained processors.

3.2 Algorithm Description

The SD algorithm has been developed as a generic algorithm that improves performance of any nonduplication-based list heuristic by selectively adding duplication into it, provided it helps in improving the performance. The algorithm (pseudocode shown in Fig. 2) completes in three main steps as described in the following sections.

3.2.1 Task Sequence Generation Phase

In this phase, an ordered task sequence using critical path-based priority is generated. It is to be noted that any of the priority scheme, as available in literature, may be used to serialize the tasks in the DAG [29], [18]. In the present work, we have used the scheme as given by Ahmad et al. [2], [18] with small modifications.

A critical path (CP), defined as the longest directed path in terms of computation and communication costs, is found in the DAG in a depth search manner. Nodes along this path are termed as CP nodes. While doing this, ties, if any, are broken on the basis of higher computation costs first and, then, (unlike Kwok's scheme which breaks the second ties randomly) on the basis of a higher number of immediate predecessors of critical path nodes, i.e., a critical path-based on maximum immediate predecessors first (CP/MIPF). Such a scheme might benefit the task graphs with multiple independent CPs, as the CP that spans more task graph nodes will be selected and those tasks will be given higher priority for scheduling in the later steps. The sequence $\{n_1, n_7, n_9\}$ constitutes the CP for the task graph of Fig. 1.

To satisfy the precedence constraints, predecessor nodes must be executed before the successor nodes. Therefore, predecessors are added in the CP sequence with priority decided on higher *b-level* and ties being broken on the basis of lower *t-level*. Nodes that are neither CP nodes nor their parents, are appended afterward using the same priority norm to get a critical path based task sequence as $\xi = \{n_1, n_3, n_2, n_7, n_6, n_5, n_4, n_8, n_9\}$ for the DAG shown in Fig. 1. The first unscheduled task in ξ is taken as the *candidate* task t_i.

3.2.2 Processor Selection Phase

In this phase, a set of suitable candidate processors (P_set) for t_i is chosen. P_set consists of all the available processors (number of processors is supplied as an input parameter) for the clique topology. Whereas, for the interconnection-constrained networks, P_set comprises of processors where the immediate predecessors of t_i are originally scheduled, plus their adjacent processors. It seems to be rational, as the left-out processors are those that are more than one hop away from the processors where the immediate predecessors of t_i

SD Algorithm

Begin

Step 1: Construct priority based task sequence
for (all tasks t_i in)
Step 2: Construct P_ set
Step 3: For each p_j in P_set, call get_EST(t_i, p_j) and record start time EST(t_i,p_j)
Step 4: Schedule t_i to p_j that gives minimum EST(t_i, p_j) and undo duplications (if any) done on any other processor in P_set
endfor

End

get_ EST (t_i, p_j)

Begin

step r1: A ← Find EST (t_i, p_j)
step r2: M_i ← Find MIIP of t_i not on p_j
step r3: If M_i does not exist
return A;
else
find first suitable slot for M_i. on p_j ;
step r4: if suitable slot exists
EFT(M_i , p_j) ← EST(M_i , p_j) + w(M_i) ;
else return A ;
step r5: If EFT(M_i , p_j) is less than A
duplicate M_i on p_j and go to step r1;
else return A ;

End

Fig. 2. Pseudocode for the SD algorithm.

were scheduled. It is felt that working with this reduced set of P will not affect the schedules much while the runtime of algorithm may benefit (on a 64 processor Hypercube network the performance degradation, as observed by us, was less than 0.1 percent, and runtime improvement was about 30 percent for the random graph suite).

3.2.3 Task Assignment and Duplication Phase

In this phase, t_i is assigned to one of the processors in P_set, which allows it to start at the earliest, using insertion-based duplication approach as follows. The task t_i can start execution on a candidate processor p_j if and only if data arrives from all of its immediate parents, so as to meet precedence constraints. The parent from which data arrives last of all is termed as the most important immediate parent (MIIP) of t_i. For a partially connected processor network, data arrival time for t_i on p_j, $DAT(t_i, p_j)$, is calculated as

$$DAT(t_i, p_j) = \max\{\min\{F_{y,k} + c_{yi} \times h_{kj}\}\} \quad \forall t_y \in pred(t_i),\ \forall p_k \in D_y. \tag{2}$$

For a fully connected network, it gets simplified as

$$DAT(t_i, p_j) = \max\{F_{y,k} + c_{yi} \times h_{kj}\} \quad \forall t_y \in pred(i), p_k = D_{y_1}. \tag{3}$$

Here, D_y corresponds to the set of processors where task t_y is scheduled/duplicated in nondecreasing order of its start time, and D_{y_i} represents its i^{th} element. This set is examined completely for interconnection constrained networks, as it is possible that a processor other than D_{y_1} (that hosts the originally scheduled t_y) may be closer to the candidate processor, thereby affecting the data arrival time of parent t_y for t_i. For a fully connected network, however, all processors are equidistant, therefore it is sufficient to consider only first processor D_{y_1} in D_y to calculate data arrival times.

Now, t_i can start execution on p_j (after the data from its MIIP (M_i) arrives), either at the ready time of the processor (p_j^R) (finish time of the last task scheduled on p_j), or at a time earlier than this if a suitable scheduling hole (time gap/slot left in a processor as a result of nonavailability of data earlier) is available on it. Suitability of a free slot is given by Definition 1.

Definition 1. *Given a set of n tasks $\{t_1, t_2 \ldots .., t_n\}$ scheduled on processor p_j, the free time slot (gap) G_k (between t_{k+1} and t_k, with G_k^S and G_k^F as its start and finish time, respectively) is suitable for task t_i if*

$$G_k^F - \max\{DAT(t_i, p_j), G_k^S\} \geq w(t_i),$$

where $G_0^S = 0, G_0^F = S_{1,j}, G_n^S = F_{n,j}$ and $G_n^F = \infty$ for $0 \leq k \leq n$, $S_{i,j}$ is the start time of t_i on p_j.

So, the earliest start time of t_i on p_j is given by (4).

$$EST(t_i, p_j) = \max\{DAT(M_i, p_j), \min\{p_j^R, G_k^S\}\}, \tag{4}$$

where G_k corresponds to the first suitable slot on p_j. For the early start of t_i, it is mandatory to reduce the data arrival time of its MIIP and, so, the algorithm duplicates M_i on p_j (if it is already not present there), provided a suitable scheduling hole exists. The earliest start time of t_i is rechecked, for resulting improvements and the process is repeated till no more MIIP or suitable-slot is available on p_j. In the process, if at any stage it is found that the duplication might increase the start time of t_i, then the duplication is not done (step r5 in Fig. 2).

Duplication is termed as redundant if the data arrival time of MIIP, with and without duplication, turns out to be the same and is avoided. Keeping in view limited resources

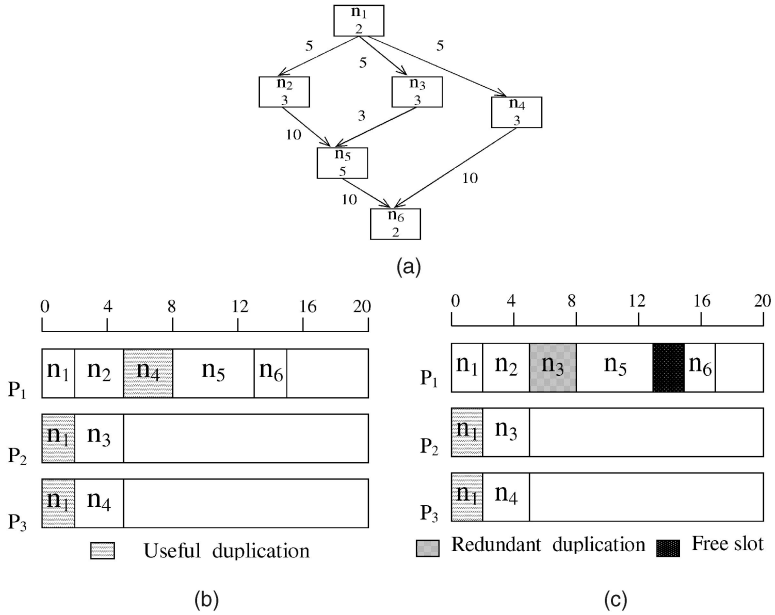

Fig. 3. Example DAG and Gantt charts illustrating the conjecture that reducing redundant duplications may help in improving the performance of an algorithm. (b) SL = 15, and (c) SL = 17.

and time complexity, the parents and ancestors of an MIIP are also not duplicated. Though, in comparison to algorithms, which try to duplicate all possible ancestors of the candidate task, the performance might suffer, but that is the *price paid* to develop an efficient algorithm within limited resources. However, we conjecture that saving of scheduling holes due to lesser duplications and prevention of redundant duplications will partly compensate for this degradation in performance, as the holes may efficiently be exploited at a later stage. We support this conjecture by means of a simple example.

3.2.4 Redundant Duplication Reduction May Benefit

Let us consider an example DAG of Fig. 3a. The corresponding task sequence ζ is given by $\{n_1, n_2, n_3, n_5, n_4, n_6\}$. The Gantt chart of schedule generated by the SD algorithm is shown in Fig. 3b. The scheduling of n_1, n_2, and n_3 is trivial. Let us see the scheduling of n_5 on P_1. The MIIP of n_5 is n_3 with data arrival time as 8. However, it is found that even if n_3 is duplicated on P_1, its data arrival time remains same as 8. The SD algorithm does not do this redundant duplication and, in turn, saves the slot between node n_2 and n_5 for future use. The scheduling of n_4 is again trivial. While scheduling n_6, it can easily be seen that the processor that gives earliest start time is P_1.

Now, the MIIP of n_6 is n_4 with data arrival time as 15. The algorithm duplicates n_4 on P_1, and in the process the saved slot is exploited to accommodate it, thereby reducing the data arrival time to 8 and giving an overall schedule length of 15. Had this slot not been saved earlier, as shown in Fig. 3c, task n_4 could not be accommodated later, resulting in a schedule length of 17. That illustrates the basis of our conjecture.

The complexity of the algorithm is calculated as follows: The task sequence in first phase of the main algorithm can be constructed in $O(v + e)$. However, this complexity comes as a part of priority selection heuristic and may be varied. The significant part of the algorithm is the *for* loop in the main algorithm that is to be executed for every task and for each processor in P_set, resulting in an overall complexity of $O(pv^2 d_{\max} + e)$ for a fully connected system.

In a recent work [27], complexity reduction schemes have been suggested for the list scheduling algorithms that limit the number of candidate processors to just two in the P_set. The same may be employed here as well to improve upon the runtime of the SD algorithm for the clique topology. For the interconnection constrained networks, the processor reduction scheme suggested in Section 3.2.2 generates a bigger list of candidate processors, but gives much better schedules as seen in the experiments conducted by us for a 64 processor Hypercube network.

A stepwise trace of the algorithm as applied to the task graph of Fig. 1 (used by Ahmad et al. [2] to compare their CPFD algorithm, which gives shortest schedule length in comparison to other algorithms like LB, BTDH, DSH, and PY) is given in Table 1. A Gantt chart for the generated schedule by SD algorithm is shown in Fig. 4 in comparison to CPFD algorithm. The SD algorithm generates the same schedule length and, at the same time, treats one duplication as redundant and saves the scheduling hole. The saving of such scheduling holes by the SD algorithm can be exploited for subsequent duplications resulting in better space and processor utilization.

3.3 Optimality of the SD Algorithm

Through the following set of theorems we prove the optimality guarantee of the SD algorithm for some restricted cases on a special class of DAGs known as Fork-Join graphs (see Fig. 5), that form the basic structure for many practical task graphs.

A Fork-Join graph is a combination of an in-tree and an out-tree graph with a root (master/source/fork) node that spawns a number of child (slave) nodes. The outgoing edge of these child nodes either culminates at an intermediate (join) node that spawns another set of child nodes or is an exit (sink) node. A general q-lobe Fork-Join DAG is shown

TABLE 1
Stepwise Trace of the SD Algorithm for the DAG of Fig. 1 (for Clique Topology)

Nodes	*s-level*	*b-level*	*t-level*	Task Sequence	step	Start times on candidate processors P_1	P_2	P_3	P_4	Node/s duplicated
n_1	12	37	0	n_1	1	0 *	0	0	0	none
n_2	8	23	6	n_3	2	2*	2	2	2	none
n_3	8	23	3	n_2	3	5	2*	2	2	n_1
n_4	9	20	3	n_7	4	8*	8	8	8	n_2
n_5	10	30	3	n_6	5	12	5*	6	6	none
n_6	5	15	10	n_5	6	12	9	2*	2	n_1
n_7	5	15	22	n_4	7	12	9	7	2*	n_1
n_8	5	15	18	n_8	8	17	14	10*	11	none
n_9	1	1	36	n_9	9	21	21	19*	21	n_7

*Selected earliest start time

in Fig. 5b. For any arbitrary kth lobe t_x^k and t_y^k represents the *fork* and *join* node, respectively, and $t_i^k (1 \leq i \leq n_k)$ as the $\mathrm{n_k}$ child nodes. The computation and communication costs in the $\mathrm{k^{th}}$ lobe are referred as $w(t_i^k)$ and $\mathrm{c_{iy}^k}$, respectively.

Theorem 1. *The SD algorithm schedules a task t_i on processor p_j at its minimum possible start time if the data arrival time of its MIIP (M_i) is optimal on this processor.*

Proof. See the appendix. □

Theorem 2. *The SD algorithm schedules any lobe of a multilobe Fork-Join DAG optimally, provided fork node of the lobe could start optimally on an empty processor.*

Proof. See the appendix. □

Corollary 1. *The SD algorithm generates optimal schedules for any single-lobe Fork-Join task graph with arbitrary computation and communication costs.*

Proof. The Fork node of any single lobe FJ DAG (Fig. 5a) is the start node of the graph as well and, hence, can always be scheduled optimally on an empty processor. Therefore, as per Theorem 2, all nodes in the lobe will be scheduled optimally.
Hence, the proof. □

Theorem 3. *The SD algorithm generates optimal schedules for any q-lobe Fork-Join task graph where 1) all child tasks in a lobe have equal sizes $(w(t_i^k) = \mu_\mathrm{k}, 1 \leq i \leq n_k)$, and 2) communication costs from child nodes to their join node are equal $(c_{iy}^k = \tau_\mathrm{k}, 1 \leq i \leq n_k)$, provided τ_k is an integral multiple of μ_k and the number of child nodes $\mathrm{n_k}$ in the $\mathrm{k^{th}}$ lobe satisfies the constraint,*

$$n_k \geq \frac{\tau_k}{\mu_k}, 1 \leq k \leq q.$$

Proof. See the appendix. □

It is to be noted that the restrictions on the number of child nodes and cost parameters is mandatory only if the communication costs (local) are more than the computation cost (local). For the task graphs with small communication delays (i.e., $\tau_i^k \leq \mu_i^k$), schedules will be optimal otherwise as well. The optimality of series parallel graphs (a more generalized form of FJ graphs) with unit communication delays has been established in literature under sufficient availability of processors [30]. The optimality of SD algorithm is, however, guaranteed with less restricted cost parameters (without any cost constraint on the fork node (w_x^k) and the links to its child nodes (c_{xi}^k) in any lobe).

Corollary 2. *The SD algorithm generates optimal schedules for any UET/UCT fork-join graph.*

Proof. For a UET/UCT task graph the ratio $\frac{\tau_k}{\mu_k}$ equals one and for any Fork-Join graph, minimum number of child nodes $n_k = 2$. Theorem 3 holds good and, hence, the proof. □

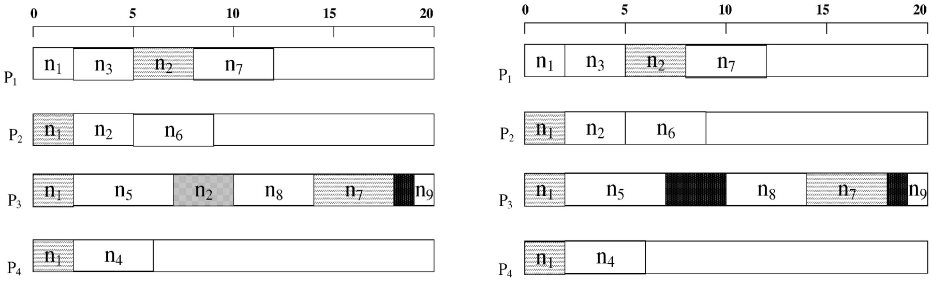

Fig. 4. Gantt charts for the generated schedule by (a) the CPFD algorithm (SL = 20 with six duplications), and by (b) the SD algorithm (SL = 20 with five duplications) for the task graph of Fig. 1.

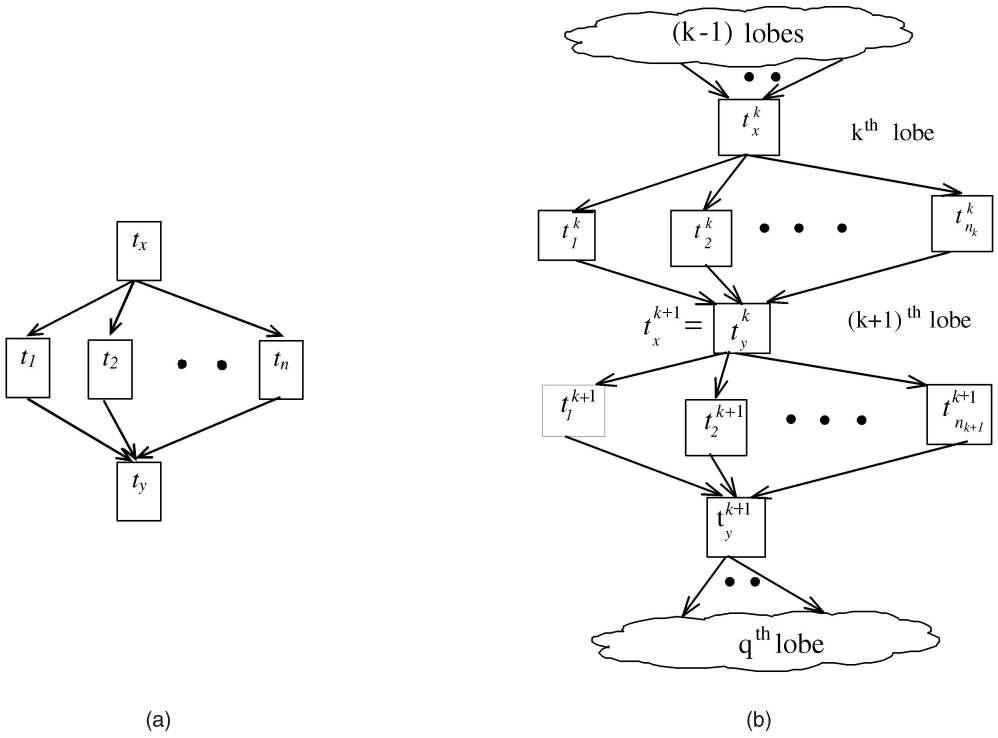

Fig. 5. (a) A single lobe fork-join DAG and (b) a q-lobe fork-join DAG.

4 Performance Results and Comparison

4.1 Performance Metrics

Performance of the SD algorithm is simulated on arbitrary random, as well as regular benchmark task graphs [17], so as to avoid any bias towards a particular graph structure. The performance metrics chosen for comparison are Normalized Schedule Length (NSL) and Efficiency that are defined as follows:

$$NSL = \frac{schedule\ length\ obtained\ by\ a\ particular\ algorithm}{maximum\ computation\ cost\ along\ a\ path\ in\ the\ DAG}.$$

The denominator gives the absolute lower bound on the schedule length.

$$Efficiency = \frac{Speedup}{Number\ of\ processors\ used} \times 100,$$

where

$$Speedup = \frac{Schedule\ length\ obtained\ on\ a\ uniprocessor\ system}{Schedule\ length\ obtained\ on\ the\ multiprocessor\ system}.$$

To have optimum utilization of resources, an algorithm giving lower NSL and higher efficiency is desirable. Duplication algorithms selected for comparison are TDS, MTDS, and GPY, and the nonduplication algorithms chosen are the best known static priority-based Modified Critical Path (MCP) algorithm [34] with $O(v^2(\log v + p))$ time complexity, and a recently proposed Topological Assignment and Scheduling Kernel (TASK) algorithm [35] that tries to refine an initial schedule based on almost similar critical path-based priority scheme [2] by incorporating efficient local search technique into it, with time complexity as $O(pv + e)$. These algorithms represent wide variety in terms of type of heuristic as well as complexity.

A comparison with the GPY algorithm (with a known performance ratio of two) helps in quantifying the performance with respect to the best available approximation algorithm. The comparison with the MTDS algorithm is interesting, as MTDS algorithm tends to solve the problem by using a clustering approach, while our algorithm tends to exploit the list based approach. Further, since results of MTDS algorithm are not available for practical DAGs, the exhaustive simulation results as presented by us will provide a quantitative overview of the improvement of MTDS over TDS algorithm for arbitrary DAGs.

4.2 Experimental Set-Up

For the sake of uniformity and to draw some conclusive results, the performance of all algorithms is evaluated for arbitrary benchmark DAGs. Although such a set-up may not result in optimal performance for TDS and MTDS algorithms, yet, it will help us in evaluating their performance on a more general and realistic platform, in comparison with other algorithms. The TDS, MTDS, and GPY algorithms have been simulated with unlimited availability of processors as they have been designed to perform under such assumptions. Algorithms using limited number of processors (SD, MCP, and TASK algorithms) were also supplied with a large number of processors initially, so that they could perform at their best. The number of processors actually used is calculated after scheduling, and is used in the efficiency results.

The random task graph suite consists of 250 graphs with 10 different sizes, varying from 50 to 500 with an increment of 50. Corresponding to each size, there are five different CCRs: 0.1, 0.5, 1.0, 2.0, and 10.0, giving wide ranging task granularities, and five different average parallelisms: 1, 2, 3,

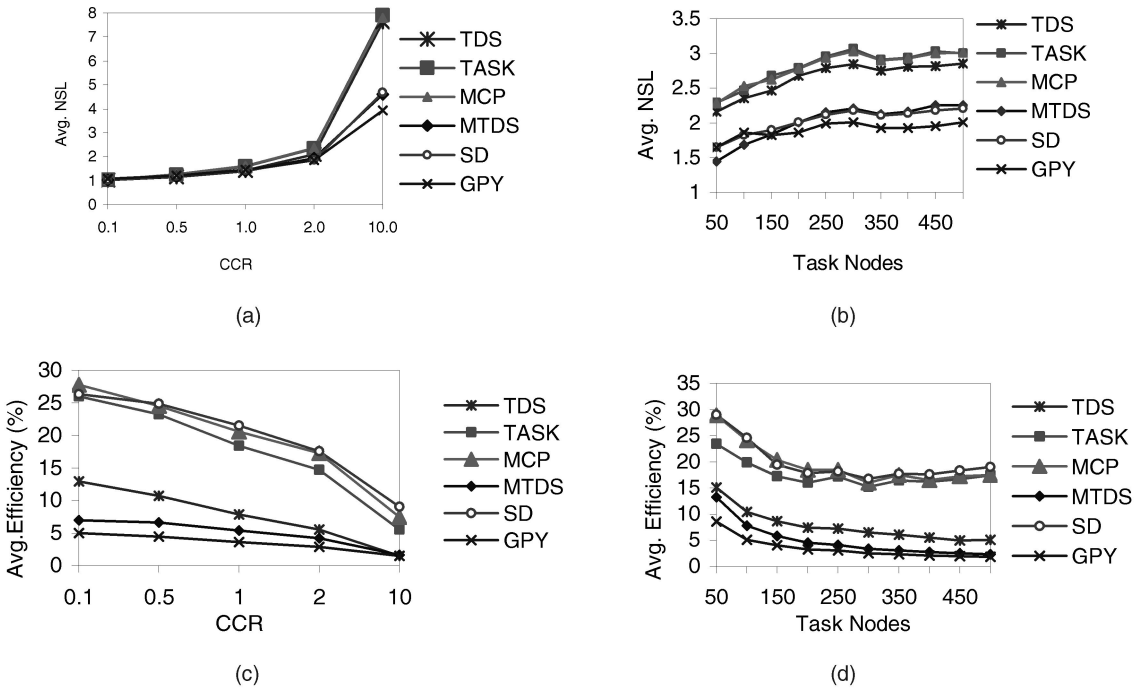

Fig. 6. Performance of the SD algorithm in terms of average NSL and average efficiency in comparison with other algorithms on random graph suite.

4, and 5 (an average parallelism of "n" implies that the average number of nonprecedence related nodes in the graph equals $n\sqrt{v}$).

The regular task graph suite also consists of 250 task graphs, corresponding to various parallel numerical applications like Gaussian Elimination (Gauss), LU decomposition (LU), Cholesky factorization (Cholesky), Laplace Equation Solver (Laplace), and Mean value analysis (MVA). As these applications operate on matrices, the matrix dimension (N) almost represents the graph size and is $O(N^2)$. The matrix dimension for the first three application graphs varies from 19 to 64, and for the later two it varies from 19 to 46, thereby giving wide representation to small as well as large size task graphs. There are 10 different sizes for every graph type and just like random graph suite, five different CCRs related to every size.

4.3 Performance Results on Random Graphs

Results for the random graph suite are presented in Fig. 6 for the clique topology for different task graph sizes and CCRs. Every result with respect to CCR is an average of 50 graphs (over 10 sizes and five average parallelisms), and with respect to graph size is an average of 25 graphs (over five CCRs and five average parallelisms).

The performance of every algorithm, as expected, tends to degrade at higher CCRs and sizes due to the resulting min-max trade offs [29], [31], [17]. However, as reflected from Figs. 6a and 6b, the duplication-based GPY, MTDS, and SD algorithms are able to cope-up effectively with this problem in comparison to the TASK, MCP, and TDS algorithms. The TDS algorithm, though based on duplication, is unable to exploit the full potential of duplication technique due to the linearity constraint that does not allow more than one parent of a candidate task to be scheduled on the same processor. The rating of algorithms on the basis of goodness of schedules generated is GPY $>$ MTDS $\approx$ SD $>$ TASK $\approx$ MCP $>$ TDS. Average NSL generated by the SD algorithm is within 1 percent and 7 percent to the MTDS and GPY algorithms, respectively.

Efficiency results (Figs. 6c and 6d) reflect the superiority of SD algorithm over all others, as it turns out to be the most efficient algorithm. The order of improvement is SD $\approx$ MCP $>$ TASK $>$ TDS $>$ MTDS $>$ GPY. Further, algorithms designed for limited processors constraint are more efficient as compared to those that work under the unbounded processors assumption. The number of processors used by the latter algorithms increases significantly with graph size, thereby limiting their efficiency and practicability for large task graphs.

Results of the SD algorithm for Hypercube network are presented in Fig. 7. Since the available duplication algorithms, as far as we know, have not been designed to take into account the interconnection constraints of the underlying network, we are presenting the results of our algorithm in comparison to a nonduplication list-based Dynamic Level Scheduling (DLS) algorithm [33] that has been shown by Kwok [17] to give quite stable performance on interconnection constrained networks.

Results shown in Figs. 7a and 7b, in comparison to the corresponding results of Fig. 6, also reflect the degradation as suffered by the algorithm due to increased communication overheads in a limited connectivity multiprocessor network. Further, as shown in Fig. 7c, the improvement in performance beyond a certain network size is negligible. This is anticipated, as the increase in network size amounts to increased diameter that ultimately results in increased communication overhead, and the problem aggravates with increasing CCRs.

4.4 Performance Results on Regular Graphs

For the regular graph suite, performance (averaged over 10 sizes $\times$ five CCRs) is shown in Fig. 8 for different graph types. Performance of an algorithm varies with the task graph structure.

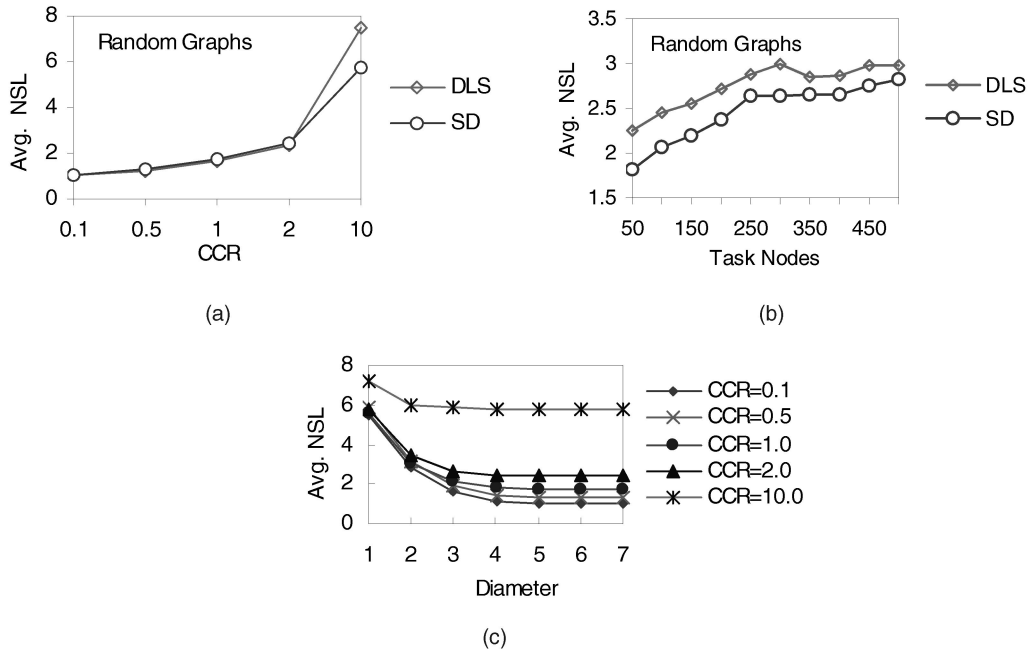

Fig. 7. Performance of the SD algorithm on the hypercube network (diameter = 6) for the random graph suite.

However, the general trend as observed for the random graph suite (in Section 4.3) is observed here as well. Schedules generated by GPY and MTDS algorithms, unlike the SD algorithm, get better at the cost of efficiency.

Table 2 summarizes the performance degradations as suffered by various algorithms for different graph types with respect to the best algorithm in that category. The SD algorithm turns out to be an efficient algorithm that generates schedules comparable to the best algorithm with remarkably less processor consumption that gets reflected in its improved efficiency results.

5 Discussion and Conclusions

A duplication-based scheduling strategy has been developed that works with limited interconnection constrained multiprocessors and tries to strike a balance between no duplication and full duplication-based heuristics. The suggested algorithm has been designed to exploit the available scheduling holes effectively, so as to generate shorter schedules without sacrificing efficiency. Duplication plays its role more effectively at high CCRs, as the formation of large sized scheduling holes increases with higher communication costs, which can be exploited to accommodate fine grain tasks conveniently. In contrast, at low CCRs, scheduling holes are small (due to lesser communication costs) and the tasks are coarse-grain, making duplication strategy more or less ineffective. Therefore, at low CCRs, the performance of the algorithm is controlled more by the choice of underlying priority selection heuristic. The improved performance of the SD algorithm is attributed to the amalgamation of the best features of priority selection heuristic along with the judicious use of the scheduling holes, by selectively duplicating the most crucial nodes and minimizing redundant duplications.

Performance comparison with some of the best-known scheduling algorithms, under the stated assumptions, shows that the SD algorithm is an efficient algorithm that generates smaller or comparable schedules with remarkably less processor consumption. Keeping it in view, the SD algorithm seems to offer a good alternative for scheduling large size task graphs in the multiprocessor systems.

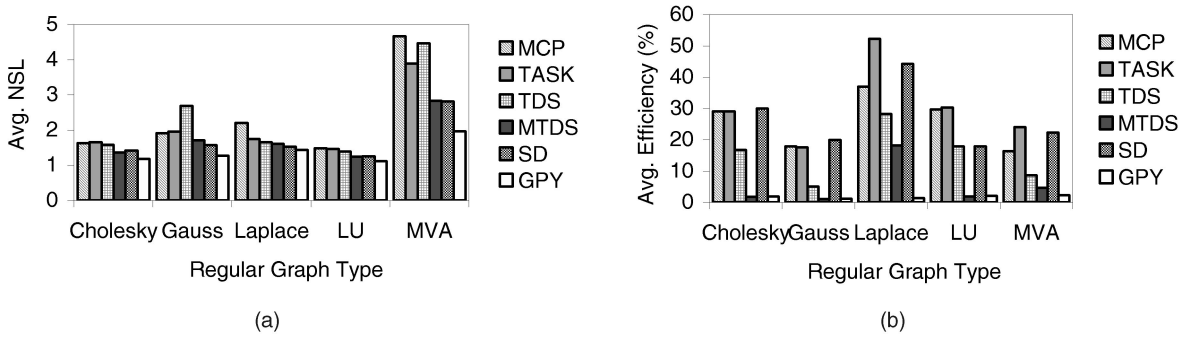

Fig. 8. Performance of the SD algorithm in comparison to other algorithms for the regular graph suite.

TABLE 2
Performance Degradation Suffered by Various Algorithms with Respect to the Best Algorithm

	(%) NSL Degradation						(%) Efficiency Degradation					
Algorithm / Graph Type	TASK	MCP	TDS	MTDS	SD	GPY	TASK	MCP	TDS	MTDS	SD	GPY
Cholesky	40	37	34	15	19	0	3	3	44	94	0	94
Gauss	54	50	111	34	24	0	12	11	75	95	0	94
Laplace	22	54	15	12	6	0	0	29	46	65	15	97
LU	31	33	25	12	13	0	0	2	41	94	41	93
MVA	98	138	128	44	44	0	0	32	64	81	7	90
Random	48	48	40	6	7	0	12	2	61	75	0	83

APPENDIX

For the sake of simplicity of notations, $\mathrm{S_i}$ and $\mathrm{F_i}$ wherever used implies the start/finish time of task $\mathrm{t_i}$ on the originally scheduled processor, i.e., D_{i_1}.

Proof of Theorem 1. We shall prove the theorem by showing that the start time of $\mathrm{t_i}$ as given by the SD algorithm cannot be improved anymore. According to the SD algorithm, (4),

$$EST(t_i, p_j) = \max\{DAT(M_i, p_j), \min\{p_j^R, G_k^S\}\}.$$

The result is obvious for $DAT(M_i, p_j) \geq \min\{p_j^R, G_k^S\}$. For $DAT(M_i, p_j) < \min\{p_j^R, G_k^S\}$ algorithm scans the free-slot list of $\mathrm{p_j}$ to look for the first suitable slot to accommodate $\mathrm{t_i}$, else the task get scheduled at the ready time of the processor (p_j^R). The task could not be started earlier than this under both the circumstances.

Hence, the proof. □

Proof of Theorem 2. We shall prove the theorem by showing that all nodes in any arbitrary kth lobe are scheduled at their minimum possible start time that cannot be reduced further.

Since the fork node, t_x^k can be scheduled optimally on an empty processor (say $\mathrm{p_e}$), so its start and finish time on $\mathrm{p_e}$ will be optimal. Let $\mathbf{S}_x^k$ and f_x^k represent optimal start and finish time of task t_x^k. The ready time of an empty processor (p_e^R) can be taken as zero in the beginning, so the earliest start time for the fork node t_x^k using (4) is,

$$EST(t_x^k, p_e) = DAT(M_x^k, p_e) = \mathbf{S_x^k}. \tag{5}$$

Now, start time of the child nodes t_i^k is limited by the data arrival time of their only MIIP t_x^k. These nodes can also start optimally at a time f_x^k (optimal finish time of their MIIP) by duplicating t_x^k on different empty processors. Let us consider scheduling of the join node t_y^k. Without loss of generality, we may assume that child nodes are arranged in nonincreasing order of time at which their data would be available to t_y^k, i.e., $F_i^k + c_{iy}^k \geq F_{i+1}^k + c_{(i+1)y}^k$. It can be easily verified that the most suitable processor (say p_s) to host t_y^k is one that hosts the MIIP t_1^k. Accordingly, we need to prove the optimality of t_y^k on this processor only.

Scheduling of t_y^k on $\mathrm{p_s}$ will zero the communication cost c_{1y}^k, thereby facilitating its earlier start. Now, the start time of t_y^k is limited by the data arrival time of its next MIIP, if it exists, else it is optimal. Since t_x^k is the only MIIP of all the child nodes, and it is already scheduled on p_s, so (4) gets modified as

$$\begin{aligned} EST(t_y^k, p_s) &= \max\{DAT(t_2^k, p_s), p_s^R\} \\ &= \max\{(F_2^k + c_{2y}^k), F_1^k\} \\ &= \max\{(f_x^k + w(t_2^k) + c_{2y}^k), (f_x^k + w(t_1^k)\} \\ &= f_x^k + \max\{(w(t_2^k) + c_{2y}^k), w(t_1^k)\}. \end{aligned} \tag{6}$$

The SD algorithm duplicates the MIIP (t_2^k) on $\mathrm{p_s}$ (if it improves the start time, i.e., $\sum_{i=1}^{2} w(t_i^k) < w(t_2^k) + c_{2y}^k$, and schedules it at its minimum possible start time on p_s (as per Theorem 1). In case the cumulative cost $\sum_{i=1}^{2} w(t_i^k) \geq w(t_2^k) + c_{2y}^k$, MIIP will not be duplicated. The process is repeated till there is no more MIIP or its duplication on $\mathrm{p_s}$ tends to deteriorate the start time of t_y^k. Thus, the start time of join node t_y^k (as given in (6)) gets generalized as,

$$EST(t_y^k, p_s) = f_x^k + \max\left\{(w(t_{m_k+1}^k) + c_{(m_k+1)y}^k), \sum_{i=1}^{m_k} w(t_i^k)\right\}, \tag{7}$$

where $\mathrm{m_k}$ is the critical number of child nodes in the kth loop whose duplication is productive, i.e.,

$$\sum_{i=1}^{m_k} w(t_i^k) < w(t_{m_k}^k) + c_{m_k y}^k,$$

but

$$\sum_{i=1}^{m_k+1} w(t_i^k) \geq w(t_{m_k+1}^k) + c_{(m_k+1)y}^k.$$

Accordingly, the finish time of the join node (t_y^k) will be given by

$$F_y^k = f_x^k + \max\left\{(w(t_{m_k+1}^k) + c_{(m_k+1)y}^k), \sum_{i=1}^{m_k} w(t_i^k)\right\} + w(t_y^k). \tag{8}$$

This cannot be improved further.

Hence the proof. □

Proof of Theorem 3. A q-lobe Fork-Join DAG is shown in Fig. 5b. We first prove that under the given cost assumptions, fork nodes in all q-lobes can be scheduled optimally

on an empty processor, and justify the constraint as stated in Theorem 3. This part is proved by induction on the number of lobes. For the first lobe (k = 1), fork node t_x^1 being the entry node, will be scheduled optimally with the schedule length given by

$$SL = F_y = f_y = f_x + \max\left\{(w(t_{m+1}) + c_{(m+1)y}), \sum_{i=1}^{m} w(t_i)\right\} + w(t_y) \quad \text{for } 1 \le \mathrm{m} \le \mathrm{n}. \tag{9}$$

The optimal finish time f_y^1 of its join node t_y^1 is thus given, by modifying (9) under the given cost assumptions as,

$$f_y^1 = f_x^1 + \max\left\{(\mu_1 + \tau_1), \sum_{i=1}^{m_1} \mu_1\right\} + w(t_y^1), \tag{10}$$

where m_1, as discussed in Theorem 2, is the critical number of child nodes, between 1 and n_1, such that

$$\sum_{\mathrm{i}=1}^{m_1} \mu_1 < \mu_1 + \tau_1 \quad \text{and} \quad \sum_{\mathrm{i}=1}^{m_1+1} \mu_1 \ge \mu_1 + \tau_1.$$

This constraint gets satisfied if,

$$\frac{\tau_1}{\mu_1} m_1 < \frac{\tau_1}{\mu_1} + 1. \tag{i}$$

Since the number of child nodes in a lobe have to be at least equal to this critical number m_1, so

$$n_1 \ge m_1. \tag{ii}$$

By combining inequalities (i) and (ii), we get $n_1 \ge \frac{\tau_1}{\mu_1}$, as stated in the theorem. The inequality (i) implies that,

$$\tau_1 \le m_1\mu_1 < \tau_1 + \mu_1 \Rightarrow \tau_1 \le \sum_{i=1}^{m_1} \mu_1 < \tau_1 + \mu_1.$$

Using the above relation in (10), we get optimal finish time of t_y^1 as

$$f_y^1 = f_x^1 + (\mu_1 + \tau_1) + w(t_y^1). \tag{11}$$

Let us assume fork node of the kth lobe can also be scheduled optimally on an empty processor. So, all other nodes in this lobe will get scheduled at their optimal time (using Theorem 2) and the optimal finish time f_y^k of its join node t_y^k will be

$$f_y^k = f_x^k + (\mu_\mathrm{k} + \tau_\mathrm{k}) + w(t_y^k). \tag{12}$$

Since the fork node of the $(\mathrm{k}+1)\mathrm{th}$ lobe is the join node of kth lobe ($t_y^k = t_x^{k+1}$), so it gets optimally scheduled as well. Accordingly, the optimal finish time of t_x^{k+1}, using (12) is

$$f_x^{k+1} = f_x^k + (\mu_\mathrm{k} + \tau_\mathrm{k}) + w(t_x^{k+1}). \tag{13}$$

Let us verify whether this fork node t_x^{k+1} can be scheduled on an empty processor (say $\mathrm{p_e}$) with the same optimal finish time or not. Now, as $p_e^R = 0$, the start time of t_x^{k+1} on $\mathrm{p_e}$ is limited by the data arrival time from its MIIP only, and any of the child nodes in kth lobe ($t_l^k, 1 \le l \le n_k$) can be its MIIP (as all of them have same computation and communication costs). The start time of fork node t_x^{k+1} on $\mathrm{p_e}$ as per (4) is given by

$$\begin{aligned} EST(t_x^{k+1}, p_e) &= DAT(M_x^{k+1}, p_e) \\ &= f_l^k + c_{ly}^k \\ &= f_x^k + w(t_l^k) + \tau_k \\ &= f_x^k + \mu_\mathrm{k} + \tau_k. \end{aligned} \tag{14}$$

Consequently, the finish time of t_x^{k+1} on $\mathrm{p_e}$ will be given by

$$F_x^{k+1} = f_x^k + \mu_\mathrm{k} + \tau_\mathrm{k} + w(t_x^{k+1}). \tag{15}$$

This is the same as the optimal finish time of f_x^{k+1} given by (13). It verifies that under the given constraints the fork node in $(\mathrm{k}+1)\mathrm{th}$ lobe can also be scheduled optimally on an empty processor.

That completes the first part of the proof.

Now, since all fork nodes $t_x^k (1 \le k \le q)$ can be scheduled optimally on an empty processor, therefore, all other nodes in the corresponding lobes, will also be scheduled optimally (using Theorem 2), and the overall makespan of the DAG will be given by $f_y^q = f_x^q + (\mu_\mathrm{q} + \tau_\mathrm{q}) + w(t_y^q)$.

That proves the theorem. □

ACKNOWLEDGMENTS

The authors are thankful to the anonymous reviewers for their useful comments, and to the All India Council for Technical Education for their Quality Improvement Program under MHRD, Government of India.

REFERENCES

[1] T.L. Adam, K.M. Chandy, and J.R. Dickson, "A Comparison of List Scheduling for Parallel Processing Systems," *Comm. ACM,* vol. 17, no. 12, pp. 685-690, Dec. 1974.

[2] I. Ahmad and Y.K. Kwok, "On Exploiting Task Duplication in Parallel Program Scheduling," *IEEE Trans. Parallel and Distributed Systems,* vol. 9, no. 9, pp. 872-892, Sept. 1998.

[3] S. Bansal, P. Kumar, and K. Singh, "A Cost-Effective Scheduling Algorithm for Message Passing Multiprocessor Systems," *Proc. 15th Int'l Conf. Parallel and Distributed Computing Systems,* pp. 47-52, Sept. 2002.

[4] S. Bansal, P. Kumar, and K. Singh, "Duplication Based Scheduling Algorithm for Interconnection Constrained Distributed Memory Machines," *Proc. Ninth Int'l Conf. High Performance Computing,* Lecture Notes in Computer Science, S. Sahni, V.K. Prasanna, and U. Shukla, eds., vol. 2552, pp. 52-62, Dec. 2002.

[5] T.L. Casavant and J.G. Kuhl, "A Taxonomy of Scheduling in General-Purpose Distributed Computing Systems," *IEEE Trans. Software Eng.,* vol. 14, no. 2, pp. 141-154, Feb. 1988.

[6] Y.C. Chung and S. Ranka, "Application and Performance Analysis of a Compile-Time Optimization Approach for List Scheduling Algorithms on Distributed-Memory Multiprocessors," *Proc. Supercomputing,* pp. 512-521, Nov. 1992.

[7] E.G. Coffman, *Computer and Job-Shop Scheduling Theory.* New York: Wiley, 1976.

[8] J.Y. Colin and P. Chrétienne, "C.P.M. Scheduling with Small Communication Delays and Task Duplication," *Operations Research,* pp. 680-684, 1991.

[9] S. Darbha and D.P. Agrawal, "Optimal Scheduling Algorithm for Distributed Memory Machines," *IEEE Trans. Parallel and Distributed Systems,* vol. 9, no. 1, pp. 87-95, Jan. 1998.

[10] S. Darbha and D.P. Agrawal, "A Task Duplication Based Scalable Scheduling Algorithm for Distributed Memory Systems," *J. Parallel and Distributed Computing,* vol. 46, no. 1, pp. 15-27, Oct. 1997.
[11] M.D. Dikaiakos, A. Rogers, and K. Steiglitz, "A Comparative Study of Heuristics for Mapping Parallel Algorithms to Message Passing Multiprocessors," technical report, Princeton Univ., 1994.
[12] M.R. Garey and D.S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness.* W.H. Freeman and Co., 1979.
[13] A. Gerasoulis and T. Yang, "A Comparison of Clustering Heuristics for Scheduling Directed Acyclic Graphs onto Multiprocessors," *J. Parallel and Distributed Computing,* vol. 16, no. 4, pp. 276-291, Dec. 1992.
[14] J.J. Hwang, Y.C. Chow, F.D. Anger, and C.Y. Lee, "Scheduling Precedence Graphs in Systems with Interprocessor Communication Times," *SIAM J. Computing,* vol. 18, no. 2, pp. 244-257, Apr. 1989.
[15] S.J. Kim and J.C. Browne, "A General Approach to Mapping of Parallel Computation upon Multiprocessor Architectures," *Proc. Int'l Conf. Parallel Processing,* vol. 2, pp. 1-8, Aug. 1988.
[16] Y.K. Kwok and I. Ahmad, "Dynamic Critical-Path Scheduling: An Effective Technique for Allocating Task Graphs to Multiprocessors," *IEEE Trans. Parallel and Distributed Systems,* vol. 7, no. 5, pp. 506-521, May 1996.
[17] Y.K. Kwok and I. Ahmad, "Benchmarking and Comparison of the Task Graph Scheduling Algorithms," *J. Parallel and Distributed Computing,* vol. 59, no. 3, pp. 381-422, Dec. 1999.
[18] Y.K. Kwok, "High Performance Algorithms for Compile Time Scheduling of Parallel Processors," PhD thesis, Hong Kong Univ. of Science and Technology, Hong Kong, May 1997.
[19] H. Kasahara and S. Narita, "Practical Multiprocessor Scheduling Algorithms for Efficient Parallel Processing," *IEEE Trans. Computers,* vol. 33, no. 11, pp. 1023-1029, Nov. 1984.
[20] W.H. Kohler, "A Preliminary Evaluation of the Critical Path Method for Scheduling Tasks on Multiprocessor Systems," *IEEE Trans. Computers,* pp. 1235-1238, Dec. 1975.
[21] B. Kruatrachue and T.G. Lewis, "Grain Size Determination for Parallel Processing," *IEEE Software,* pp. 23-32, Jan. 1988.
[22] A. Munier and C. Hanen, "Using Duplication for Scheduling Unitary Tasks on *m* Processors with Unit Communication Delays," *Theoretical Computing Science,* 1997.
[23] M.A. Palis, J.C. Liou, and D.S.L. Wei, "Task Clustering and Scheduling for Distributed Memory Parallel Architectures," *IEEE Trans. Parallel and Distributed Systems,* vol. 7, no. 1, pp. 46-55, Jan. 1996.
[24] C.H. Papadimitriou and M. Yannakakis, "Towards an Architecture-Independent Analysis of Parallel Algorithms," *SIAM J. Computing,* vol. 19, no. 2, pp. 322-328, Apr. 1990.
[25] C.I. Park and T.Y. Choe, "An Optimal Scheduling Algorithm Based on Task Duplication," *IEEE Trans. Computers,* vol. 51, no. 4, pp. 444-448, Apr. 2002.
[26] G.L. Park, B. Shirazi, and J. Marquis, "DFRN: A New Approach for Duplication Based Scheduling for Distributed Memory Multiprocessor Systems," *Proc. 11th Int'l Parallel Processing Symp.,* pp. 157-166, Apr. 1997.
[27] A. Radulescu and A.J.C. van Gemund, "Low Cost Task Scheduling for Distributed-Memory Machines," *IEEE Trans. Parallel and Distributed Systems,* vol. 13, no. 6, pp. 648-658, June 2002.
[28] H.El. Rewini and T.G. Lewis, "Scheduling Parallel Programs onto Arbitrary Target Machines," *J. Parallel and Distributed Computing,* vol. 9, no. 2, pp. 138-153, June 1990.
[29] H.El. Rewini, T.G. Lewis, and H.H. Ali, *Task Scheduling in Parallel and Distributed Systems.* New Jersey: Prentice Hall, 1994.
[30] M.W. Schaffter, "Scheduling Jobs with Communication Delays: Complexity Results and Approximation Algorithms," PhD thesis, Technical Univ. of Berlin, Germany, 1996.
[31] S. Sevalkumar and C.V. Ramamoorthy, "Scheduling Precedence Constrained Task Graphs with Non-Negligible Inter-Task Communication onto Multiprocessors," *IEEE Trans. Parallel Distributed Systems,* pp. 328-336, Mar. 1994.
[32] B. Shirazi, M. Wang, and G. Pathak, "Analysis and Evaluation of Heuristic Methods for Static Task Scheduling," *J. Parallel and Distributed Computing,* vol. 10, no. 3, pp. 222-232, 1990.
[33] G.C. Sih and E.A. Lee, "A Compile-Time Scheduling Heuristic for Interconnection-Constrained Heterogeneous Processor Architectures," *IEEE Trans. Parallel and Distributed Systems,* vol. 4, no. 2, pp. 75-87, Feb. 1993.
[34] M.Y. Wu and D.D. Gajski, "Hypertool: A Programming Aid for Message-Passing Systems," *IEEE Trans. Parallel and Distributed Systems,* vol. 1, no. 3, pp. 330-343, July 1990.
[35] M.Y. Wu, W. Shu, and J. Gu, "Efficient Local Search for DAG Scheduling," *IEEE Trans. Parallel and Distributed Systems,* vol. 12, no. 6, pp. 617-627, June 2001.
[36] T. Yang and A. Gerasoulis, "DSC: Scheduling Parallel Tasks on an Unbounded Number of Processors," *IEEE Trans. Parallel and Distributed Systems,* vol. 5, no. 9, pp. 951-967, Sept. 1994.
[37] T. Yang, "Scheduling and Code Generation for Parallel Architectures," PhD thesis, Rutgers Univ., May 1993.

Savina Bansal (S'01) received the BE degree (electronics and electrical communications) from Punjab Engineering College, Chandigarh in 1988, and the ME degree in computer science and engineering from TIET, Patiala in 1994. Presently, she is pursuing doctoral research in the Department of Electronics and Computer Engineering at the Indian Institute of Technology Roorkee, India. She is on study leave (under the Quality Improvement Program of AICTE under MHRD, Government of India) from her parent institute GZS College of Engineering and Technology, Bathinda (Punjab), where she is on the rolls of faculty of Department of Electronics Engineering since 1993. Her areas of research include fault tolerant interconnection networks, parallel and distributed computing, and multiprocessor scheduling. She is a student member of the IEEE and the IEEE Computer Society.

Padam Kumar (M'03) received the BTech and MTech degrees in electrical engineering from the Indian Institute of Technology, Delhi, India and, subsequently, the PhD degree in computer science from the University of Roorkee (now, Indian Institute of Technology, Roorkee). He has been with the Department of Electronics and Computer Engineering, University of Roorkee since 1974, and is currently a professor of computer science and engineering. His current areas of interests are computer architectures, multiprocessors, fault tolerant computing, and real time multiprocessor systems. He is a member of the IEEE.

Kuldip Singh received the bachelor and master's degrees in electronics and communication engineering, and the PhD degree in computer science, all from the University of Roorkee. He joined the University of Roorkee (now, IIT Roorkee) in 1973 as a lecturer. Presently, he is a professor in computer engineering at IIT Roorkee. His areas of interests are parallel processing and computer communication networks. Dr. Singh has published more than 40 technical papers in journals and conference proceedings and organized several national and international conferences/workshops. He is also the founder chairman of the editorial board of *Indian Journal of Continuing Engineering Education*.